

Assignment 1

Name: Zuha Aqib

ERP ID: 26106

Date: 22nd February 2026

Course: Introduction to Blockchain

Table of Contents

Question 1	3
Diagram	4
Question 2.....	5
Part A.....	5
Part B.....	5
Question 3.....	6
Part A.....	6
Part B.....	7
Question 4.....	8
Part A.....	8
Diagram	9
Part B.....	10
Diagram	11
Question 5.....	12
Part A.....	12
Part B.....	13
Question 6.....	14
Part A.....	14
Part B.....	14
Question 7	15
Part A.....	15
Part B.....	17
Part C	19

Part D	21
Question 8.....	23
Part A.....	23
Part B.....	23
Question 9.....	24
Part A.....	24
Part B.....	24
Part C	25
Question 10.....	26
Part A.....	26
Part B.....	26

Question 1

Suppose Alice wants to send a confidential message to Bob. She does not want anyone else to see what she sent. Alice also wants Bob to be sure it really came from her. How can we meet all the requirements? Explain by specifying all the actions taken by both Alice and Bob. Feel free to simply draw a block diagram.

We need two things: confidentiality (only Bob can read it) and authentication/integrity (Bob can confirm it came from Alice and wasn't changed). For confidentiality, Alice should use public-key encryption: she encrypts using Bob's public key, and only Bob can decrypt using his private key. For authenticity and integrity, Alice uses a digital signature: she hashes the message and signs the hash with her private key; Bob verifies the signature using Alice's public key. The common safe order is sign first, then encrypt, so the signature is also protected and the whole package stays private.

Key setup (already exists):

- Alice has: (Alice_private, Alice_public)
- Bob has: (Bob_private, Bob_public)

Actions by Alice

1. Write message M
2. Compute $h = \text{Hash}(M)$
3. Create signature $S = \text{Sign}(h, \text{Alice_private})$
4. Encrypt the bundle with Bob's public key: $C = \text{Encrypt}((M, S), \text{Bob_public})$
5. Send C to Bob

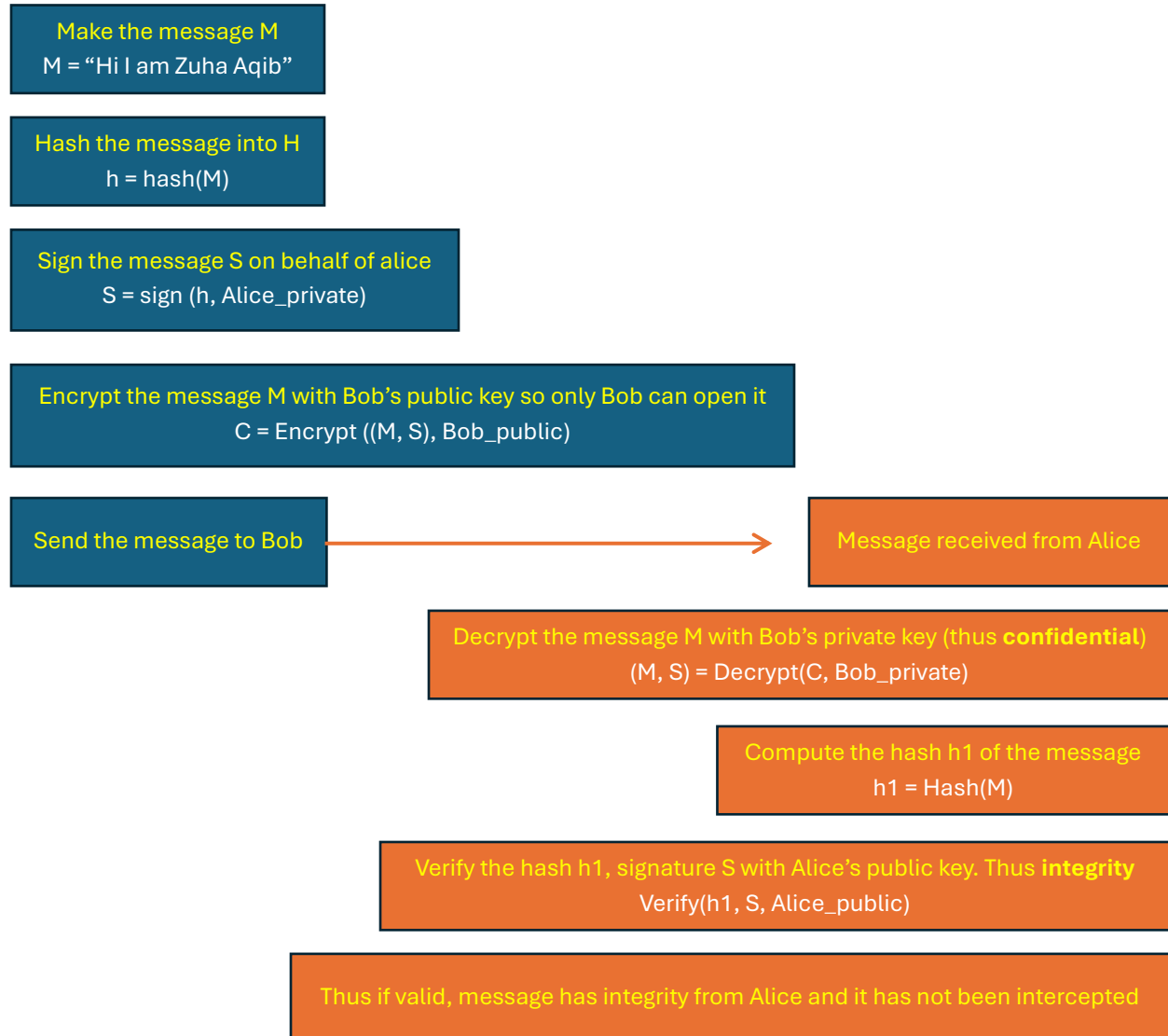
Actions by Bob

1. Decrypt: $(M, S) = \text{Decrypt}(C, \text{Bob_private})$
2. Compute $h1 = \text{Hash}(M)$
3. Verify: $\text{Verify}(h1, S, \text{Alice_public})$
4. If valid $\rightarrow$ message is private (only Bob decrypted) and truly from Alice (signature matches)

Diagram

Alice

Bob



Question 2

Bob receives a message that claims to be signed by Alice. He tries to verify it using Alice's public key, but the verification fails.

Part A

a) What does this outcome mean?

If verification fails using Alice's public key, it means Bob cannot trust the signature. In simple terms: the message is not proven to be from Alice, and/or the message is not exactly the same as what was signed. So Bob should treat it as unauthentic and possibly tampered.

Part B

b) Based on this, what possible issues or attacks should Bob suspect?

Bob should suspect these common possibilities:

- Message was modified in transit (tampering):
Someone changed even 1 character after Alice signed it, so the hash/signature no longer matches.
- It was not signed by Alice at all (forgery):
An attacker (or someone else) sent a message claiming "Alice signed it" but they don't have Alice's private key, so the signature is invalid.
- Bob is using the wrong public key for Alice (key mismatch / key substitution attack):
Bob may have the wrong Alice public key saved, or an attacker replaced Alice's public key with their own (classic man-in-the-middle / key replacement scenario).
- Wrong verification details / formatting changes:
Sometimes verification fails if Bob verifies a slightly different version of the message than what was signed (e.g., extra spaces/newlines, different encoding, different file content). Even tiny formatting differences break signatures.

What Bob should do next:

- Re-check he's using the correct Alice public key from a trusted source/certificate.
- Ask Alice to resend the message/signature through a trusted channel.
- Treat the current message as untrusted and do not act on it.

Question 3

Part A

a) Alice comes up with her own digital signature scheme. She writes a message and encrypts that message using her Private key. She then sends this message and the encrypted message both to Bob. Can Bob be sure that this message came from Alice?

Alice writes a message M, “encrypts” it using her private key (call the result E), and sends both M and E to Bob.

Can Bob be sure the message came from Alice?

Not really / not safely. Bob can try to check by using Alice’s public key to “decrypt” E and see if it matches M. If it matches, it looks like only Alice could have produced E (because only Alice should have the private key).

But this is not a proper digital signature scheme. Digital signatures are not done by encrypting the whole message with a private key. Real signature methods sign a hash of the message using a proper signing algorithm. If Alice invents her own method like this, it can have hidden weaknesses (wrong padding, replay issues, efficiency problems, etc.). So Bob should not consider it a reliable/standard proof.

Bob might get a match, but he still should not be fully confident because the scheme is not a correct/standard signature design.

Part B

b) Alice modifies this scheme. She writes a message and encrypts that with her private key (as in the previous part). She then hashes the encrypted message and sends this hash along with the original message to Bob. Can Bob be sure that this message is from Alice?

Now Alice again encrypts the message with her private key to get E , then computes $H = \text{Hash}(E)$, and sends H along with the original message M to Bob.

Can Bob be sure the message is from Alice?

No. A hash value by itself does not prove who created it. Also, Bob only has M and H . He does not have E , so he can't recompute $\text{Hash}(E)$ to compare with H . Even worse, anybody could send some message M and attach some random hash value H — there's no verification using Alice's public key happening here.

Bob cannot be sure the message came from Alice in this modified scheme.

The correct idea would be:

$S = \text{Sign}(\text{Hash}(M), \text{Alice_private})$ and Bob verifies using Alice_public .

Question 4

In class we learnt that a symmetric key is often used for communication in-bulk i.e., when there is a lot of data that must be transferred. This is done because symmetric keys operate a lot faster than asymmetric keys.

Part A

a) Assume that Alice and Bob share a secret, symmetric key. Devise a scheme with which Alice can send a plain-text message to Bob and ensure that Bob can verify the authenticity of this message. Assume that Alice and Bob must share a LOT of data and asymmetric keys are not viable.

Since Alice and Bob already share a secret symmetric key K , they can use that same key to get both privacy and authenticity without using any asymmetric keys. A standard way to do this is to use encryption for confidentiality and a MAC (Message Authentication Code) for authenticity and integrity. The idea is that a MAC is like a “secret stamp” that only someone who knows K can generate, so if Bob can recompute the same MAC, he knows the message really came from someone with the shared key and it was not modified.

A clean scheme is Encrypt-then-MAC: Alice encrypts the plaintext message using K to get ciphertext C , then computes a MAC tag T over C using the same key K , and sends (C, T) to Bob. Bob first recomputes the MAC on the received ciphertext and checks if it matches T ; if it matches, he decrypts C to recover the message. If the MAC check fails, he rejects the message as forged or tampered.

Steps (Encrypt-then-MAC):

- Alice:
 - $C = \text{Enc}(K, M)$,
 - $T = \text{MAC}(K, C)$,
 - send (C, T)
- Bob:
 - compute $T' = \text{MAC}(K, C)$.
 - If $T' \neq T$ reject;
 - else decrypt $M = \text{Dec}(K, C)$

Diagram

Alice

Bob

Take plaintext message **M**

Encrypt it using the shared key: **C = Enc(K, M)**

Compute a MAC over the ciphertext: **T = MAC(K, C)**

Send **(C, T)** to Bob

Receive **(C, T)** from Alice

Recompute MAC on the received ciphertext: **T' = MAC(K, C)**

If **T' ≠ T**, reject (message was forged or altered)

If it matches, decrypt: **M = Dec(K, C)**

Why this gives authenticity:

Only someone who knows the secret key **K** can produce a valid MAC tag **T**, so Bob can trust the sender knew **K** (so it's authentic) and the data wasn't modified.

Part B

b) Consider a scenario in which Alice wishes to relay to Bob the message “Give me 3.50\$”. A malicious actor, Kevin, duplicates this message and sends it twice to Bob (this is called a replay attack). According to the scheme you devised in the previous part, what will the outcome be? Further, devise a scheme in which duplicated messages are rejected.

With the scheme above, a replay attack is still possible. If Kevin copies a valid transmitted pair (C, T) for the message “Give me 3.50\$” and sends the same pair again later, Bob will recompute the MAC and it will still match (because the ciphertext is identical), so Bob will accept it again and decrypt it again. In other words, the basic Encrypt-then-MAC scheme by itself does not detect duplicates, so Bob would process the same request twice.

To reject duplicated messages, Alice must include a fresh/unique value inside each message that Bob can check, such as a sequence number (counter) or a nonce (random value used once). A practical solution for bulk communication is a sequence number: Alice attaches a counter i to the message before encrypting and MACing, and Bob keeps track of the last accepted counter. If Bob ever receives a message with an old counter value again, he rejects it as a replay.

Steps (Anti-replay with sequence number):

- **Alice:**
 - maintain counter $i = 1, 2, 3, \dots$
 - Create payload $P = (i \parallel M)$.
 - Encrypt $C = \text{Enc}(K, P)$.
 - Compute $T = \text{MAC}(K, C)$.
 - Send (C, T) .
- **Bob:**
 - verify MAC first. If valid, decrypt to get (i, M) .
 - If i is not new (e.g., $i \leq \text{last accepted value}$), reject as a replay;
 - otherwise accept and update the stored counter.

This way, Kevin can duplicate the ciphertext, but Bob will notice the repeated sequence number and reject the second copy.

Diagram

Old design = purple & green. New design = orange.

Alice

Bob

To stop replays, Alice must include something **fresh/unique** in each message that Bob can check, such as a **nonce** (random number used once) or a **sequence number** (counter).

Maintain counter i (1,2,3,...)

Take plaintext message M

Create payload $P = (i || M)$

Encrypt it using the shared key: $C = \text{Enc}(K, P)$

Compute a MAC over the ciphertext: $T = \text{MAC}(K, C)$

Send (C, T) to Bob

Receive (C, T) from Alice

Recompute MAC on the received ciphertext: $T' = \text{MAC}(K, C)$

If $T' \neq T$, reject (message was forged or altered)

If it matches, decrypt to get $P = (i, M)$: $P = \text{Dec}(K, C)$

If i is **not greater than** the last accepted i (or not equal to expected), **reject as replay**

Otherwise accept and update stored i

Now if Kevin replays the old message, it has the old i , so Bob rejects it.

Question 5

In a Bitcoin blockchain, a transaction claims “Alice pays 0.3 BTC to Bob.”

Part A

a) What checks must be performed on Alice’s digital signature before this transaction is considered valid?

For the transaction “Alice pays 0.3 BTC to Bob” to be valid, nodes/miners don’t just trust the sentence - they validate the signature proof that Alice is allowed to spend the coins. In Bitcoin, Alice is spending some previous output (a UTXO) that was locked to her, and she must provide a valid signature to “unlock” it.

The main signature checks are:

- Signature verifies against the transaction data:
The signature must correctly verify on the transaction’s hash (the exact transaction contents that are being authorized, including outputs like “0.3 BTC to Bob” and the input being spent). If anything in the signed parts is changed, verification fails.
- Correct public key is provided:
Alice must provide the public key that matches the UTXO she is spending (i.e., the previous output was locked to a public key hash / script that corresponds to Alice). The public key must match what the locking script requires.
- Signature matches that public key:
Using Alice’s public key, the network checks the cryptographic verification result is true (i.e., the signature could only have been produced by the holder of the corresponding private key).
- Signature is well-formed under Bitcoin rules:
The signature must be in an acceptable encoding/format and satisfy standard validity rules (e.g., not malformed). (In practice Bitcoin also has strictness rules to avoid weird encodings and malleability issues.)

If these pass, the signature part is considered valid, meaning Alice authorized spending that specific input in that specific transaction.

Part B

b) Could Alice later deny sending it? Why or why not?

Cryptographically, no. A valid Bitcoin signature is treated as proof that the sender had the private key for the coins being spent, and the signature is bound to the exact transaction contents. So if the signature verifies, Alice cannot realistically claim “I didn’t authorize this” in a cryptographic sense, because only the private key holder could have produced that signature.

The only “argument” she could try to make is non-cryptographic, like: “my private key was stolen / my wallet was compromised.” But that doesn’t invalidate the transaction on the blockchain - it just explains how someone else might have signed using her key.

Question 6

Consider a blockchain with three blocks. Block 1 contains transactions and has a hash H1 = ABC123. Block 2 contains its own transactions and includes H1 in its data before being hashed to produce H2 = DEF456. Block 3 does the same, resulting in H3 = GHI789.

Part A

a) If a transaction in Block 1 is altered, explain why merely recalculating its hash to H1' is insufficient to hide the tampering.

In a blockchain, each block's hash depends on all the data inside that block, and also (from Block 2 onward) includes the previous block's hash as part of its own data. So if a transaction in Block 1 is changed, the hash of Block 1 changes from H1 to H1'. But Block 2 was originally created using the old value H1 = ABC123 inside its data. That means Block 2's stored "previous hash" no longer matches the new hash of Block 1. Even if the attacker recalculates Block 1's hash to get H1', everyone can still see the chain is broken because Block 2 still points to the old hash ABC123. So recalculating only Block 1's hash does not hide tampering - it creates a mismatch with Block 2.

Part B

b) What must an attacker do to make the altered chain appear valid to a node synchronizing from the latest block? Why does Proof-of-Work make this practically very difficult?

To make the altered chain look valid, the attacker would have to update every later block so the links match again. Concretely, after changing Block 1, they must: put H1' into Block 2 and re-hash Block 2 to get a new H2'; then put H2' into Block 3 and re-hash Block 3 to get H3', and so on for all following blocks. In a Proof-of-Work blockchain, this is not just "recalculate a hash once" - each block's hash must satisfy a difficulty target (the mining puzzle), which requires doing a huge amount of trial-and-error hashing to find a valid nonce. So the attacker would need to re-mine Block 2, Block 3, ... and catch up to (and surpass) the honest chain while the rest of the network continues mining new blocks. This becomes practically very difficult unless the attacker controls a massive amount of the total mining power, because the honest network keeps extending the real chain faster than the attacker can rebuild the fake one.

Question 7

Consider as simple encryption procedure using a public/private key pair. You have the public key as the integer $p=7$ and the “secret” private key as the integer $s=3$. The method to encrypt an integer m is: $c = m^p \bmod 33$ And the method to decrypt an encrypted integer is: $m = c^s \bmod 33$ Assume that the input characters have the following numerical values: $a/A = 0, b/B = 1, c/C = 2, \dots, z/Z = 25$.

Part A

a) Encrypt the word ‘SurpriseQuiz’ and write down the sequence of numbers.

I DID THIS VIA CODE. NOTEBOOK IS ATTACHED

First convert letters to numbers:

“Surprise”: $S = 18, u = 20, r = 17, p = 15, r = 17, i = 8, s = 18, e = 4,$

“Quiz”: $q = 16, u = 20, i = 8, z = 25$

Now encrypt each using $c = m^7 \bmod 33$. The encrypted sequence is:

6 26 8 27 8 2 6 16 25 26 2 31

Step 1) Letter -> number mapping (a=0 ... z=25):

S -> 18
u -> 20
r -> 17
p -> 15
r -> 17
i -> 8
s -> 18
e -> 4
Q -> 16
u -> 20
i -> 8
z -> 25

Step 2) Encrypt each number: $c = m^p \bmod n$

S: $18^7 \bmod 33 = 612220032 \bmod 33 = 6$
u: $20^7 \bmod 33 = 1280000000 \bmod 33 = 26$
r: $17^7 \bmod 33 = 410338673 \bmod 33 = 8$
p: $15^7 \bmod 33 = 170859375 \bmod 33 = 27$
r: $17^7 \bmod 33 = 410338673 \bmod 33 = 8$
i: $8^7 \bmod 33 = 2097152 \bmod 33 = 2$
s: $18^7 \bmod 33 = 612220032 \bmod 33 = 6$
e: $4^7 \bmod 33 = 16384 \bmod 33 = 16$
Q: $16^7 \bmod 33 = 268435456 \bmod 33 = 25$
u: $20^7 \bmod 33 = 1280000000 \bmod 33 = 26$
i: $8^7 \bmod 33 = 2097152 \bmod 33 = 2$
z: $25^7 \bmod 33 = 6103515625 \bmod 33 = 31$

Final encrypted sequence:

6 26 8 27 8 2 6 16 25 26 2 31

Part B

b) Decrypt the sequence “1 2 13 29 20 2 7 20 21 16 8 16 13 28”. Write down the characters and what you think of them.

I DID THIS VIA CODE. NOTEBOOK IS ATTACHED

“1 2 13 29 20 2 7 20 21 16 8 16 13 28”

Decrypt each using $m = c^3 \bmod 33$, then map back to letters:

Decrypted numbers → letters:

$1 = b, 8 = i, 19 = t, 2 = c, 14 = o, 8 = i, 13 = n, 14 = o, 21 = v, 4 = e, 17 = r, 4 = e, 19 = t, 7 = h$

So the text becomes:

“bitcoinovereth” (i.e., “bitcoin over eth”).

It actually looks like a meaningful phrase, not random letters.

Step 1) Decrypt each number: $m = c^s \bmod n$

$1^3 \bmod 33 = 1 \bmod 33 = 1$
 $2^3 \bmod 33 = 8 \bmod 33 = 8$
 $13^3 \bmod 33 = 2197 \bmod 33 = 19$
 $29^3 \bmod 33 = 24389 \bmod 33 = 2$
 $20^3 \bmod 33 = 8000 \bmod 33 = 14$
 $2^3 \bmod 33 = 8 \bmod 33 = 8$
 $7^3 \bmod 33 = 343 \bmod 33 = 13$
 $20^3 \bmod 33 = 8000 \bmod 33 = 14$
 $21^3 \bmod 33 = 9261 \bmod 33 = 21$
 $16^3 \bmod 33 = 4096 \bmod 33 = 4$
 $8^3 \bmod 33 = 512 \bmod 33 = 17$
 $16^3 \bmod 33 = 4096 \bmod 33 = 4$
 $13^3 \bmod 33 = 2197 \bmod 33 = 19$
 $28^3 \bmod 33 = 21952 \bmod 33 = 7$

Step 2) Number -> letter mapping (0=a ... 25=z):

1 -> b
8 -> i
19 -> t
2 -> c
14 -> o
8 -> i
13 -> n
14 -> o
21 -> v
4 -> e
17 -> r
4 -> e
19 -> t

19 -> t
7 -> h

Final decrypted text: bitcoinovereth

Part C

c) Suppose Alice wants to transmit the word “SaveMe”. What is the final message that Alice transmits?

I DID THIS VIA CODE. NOTEBOOK IS ATTACHED

1. Convert “SaveMe” → numbers

S=18, a=0, v=21, e=4, M=12, e=4

So message list: [18, 0, 21, 4, 12, 4]

2. Compute list-hash input:

$$\sum = 18 + 0 + 21 + 4 + 12 + 4 = 59$$

$$x = 59 \bmod 50 = 9$$

3. Hash:

$$h(9) = [e^9 + 100] \bmod 32 = 11$$

4. Signature (private key exponent $s = 3$):

$$sig = 11^3 \bmod 33 = 11$$

Final message Alice transmits: **18 0 21 4 12 4 11**

Plaintext: SaveMe

Step 1) Letter -> number (a=0 ... z=25):

S -> 18

a -> 0

v -> 21

e -> 4

M -> 12

e -> 4

Step 2) List-hash input:

sum(nums) = 59

$x = \text{sum}(\text{nums}) \bmod 50 = 59 \bmod 50 = 9$

Step 3) Hash computation:

$h(9) = \text{floor}(e^9 + 100) \bmod 32 = 11$

Step 4) Signature (private-key encryption of hash):

$\text{sig} = 11^3 \bmod 33 = 1331 \bmod 33 = 11$

Final transmitted message (m1 m2 ... mn sig):

18 0 21 4 12 4 11

Part D

d) You receive a message “12 20 18 19 0 5 0 17 14 2 10 18 1 0 2 7 4 18 7 14 2 10 18 5”. It is known that the scheme in part (c) is used. Is this message from Alice? Why or why not? If it is a message from Alice then write the message.

I DID THIS VIA CODE. NOTEBOOK IS ATTACHED

Received:

12 20 18 19 0 5 0 17 14 2 10 18 1 0 2 7 4 18 7 14 2 10 18 5

Here last number is the signature:

- Message numbers:

[12, 20, 18, 19, 0, 5, 0, 17, 14, 2, 10, 18, 1, 0, 2, 7, 4, 18, 7, 14, 2, 10, 18]

- Given $sig = 5$

1. Compute sum:

$$\Sigma = 218 \Rightarrow x = 218 \bmod 50 = 18$$

2. Compute hash:

$$h(18) = [e^{18} + 100] \bmod 32 = 5$$

3. Expected signature using Alice's private key:

$$sig_{expected} = 5^3 \bmod 33 = 26$$

But received signature is 5, not 26.

Conclusion: No, this message is NOT from Alice under this scheme (signature does not match), meaning it was forged or modified.

(Extra check: verifying with public key also fails: $5^7 \bmod 33 \neq 5$.)

```

Received message:
 12 20 18 19 0 5 0 17 14 2 10 18 1 0 2 7 4 18 7 14 2 10 18 5
-----
Step 1) Separate message numbers and signature:
msg_nums = [12, 20, 18, 19, 0, 5, 0, 17, 14, 2, 10, 18, 1, 0, 2, 7, 4, 18, 7, 14, 2, 10, 18]
sig = 5
-----
Step 2) Compute hash of message list:
sum(msg_nums) = 218
x = 218 mod 50 = 18
h(18) = floor(e^18 + 100) mod 32 = 5
-----
Step 3) Expected signature (using private key rule):
sig_expected = 5^3 mod 33 = 125 mod 33 = 26
-----
Step 4) Compare signatures:
received sig = 5
expected sig = 26
MATCH? False
-----
Step 5) Public-key verification check (sig^p mod 33 == hash):
sig^p mod 33 = 5^7 mod 33 = 78125 mod 33 = 14
hash (hx)     = 5
VERIFIED? False
-----
Step 6) Decode message numbers -> letters:
decoded = mustafarocksbacheshocks

FINAL: This is NOT from Alice under this scheme (signature check fails).

```

Question 8

Part A

a) What is the fundamental difference between Symmetric and Asymmetric (Public Key) cryptography?

The fundamental difference is in the keys used. In symmetric cryptography, the same secret key is used for both encryption and decryption, so both parties must already share that key and keep it private. In asymmetric (public-key) cryptography, each person has a key pair: a public key that can be shared with everyone and a private key that must be kept secret. What one key does, only the other can undo (for example, encrypt with someone's public key and only their private key can decrypt), and the private key can also be used to create digital signatures that others verify using the public key.

Part B

b) For the following goals, state whether symmetric or asymmetric cryptography is typically used. Also state your reasoning: - i) Encrypting a large file for storage. - ii) Proving your identity to a website. - iii) Establishing a secure connection to your bank's server.

i) Encrypting a large file for storage → Symmetric

Symmetric encryption is typically used because it is much faster and more efficient for large amounts of data. Public-key encryption is usually too slow for encrypting big files directly.

ii) Proving your identity to a website → Asymmetric

This is typically done using public-key cryptography (digital signatures / certificates). You prove identity by demonstrating you control a private key (for example, in certificate-based authentication), and the website can verify using the corresponding public key.

iii) Establishing a secure connection to your bank's server → Both (hybrid, but mainly asymmetric first)

In practice (like in TLS/HTTPS), asymmetric cryptography is used at the start to authenticate the server and securely agree on a session key, and then symmetric cryptography is used for the actual data transfer because it is fast. So it's a hybrid: asymmetric to set up trust + symmetric for bulk encrypted communication.

Question 9

A simple hash function is defined as $H(x) = (5x + 3) \bmod 16$.

Part A

a) Calculate the hash of the following numbers: $H(7)$, $H(23)$, $H(39)$.

1) $H(7)$

$$5(7) + 3 = 35 + 3 = 38$$

$$38 \bmod 16: 16 \times 2 = 32, \text{ remainder } 38 - 32 = 6$$

$$\Rightarrow H(7) = 6$$

2) $H(23)$

$$5(23) + 3 = 115 + 3 = 118$$

$$118 \bmod 16: 16 \times 7 = 112, \text{ remainder } 118 - 112 = 6$$

$$\Rightarrow H(23) = 6$$

3) $H(39)$

$$5(39) + 3 = 195 + 3 = 198$$

$$198 \bmod 16: 16 \times 12 = 192, \text{ remainder } 198 - 192 = 6$$

$$\Rightarrow H(39) = 6$$

Part B

b) What do you observe about the results for $H(7)$ and $H(23)$? Explain why this happens, relating it to the properties of the modulo operation.

We observe that $H(7) = H(23)$ even though 7 and 23 are different. This happens because 23 and 7 differ by 16: $23 = 7 + 16$

With modulo 16, adding 16 doesn't change the remainder in the final result. More clearly:

$$H(7 + 16) = (5(7 + 16) + 3) \bmod 16 = (5 \cdot 7 + 5 \cdot 16 + 3) \bmod 16$$

Since $5 \cdot 16 = 80$ and $80 \bmod 16 = 0$, that extra part disappears under mod 16, so the hash stays the same. That's why inputs spaced by 16 can collide here.

Part C

c) Based on this, what is a major weakness of this hash function?

A major weakness is that it produces lots of collisions and is very predictable.

- The output range is only 0 to 15 (because mod 16), so many different inputs must map to the same output.
- Also, because it's a simple linear formula with a small modulus, it's easy to find different values of x that give the same hash (like 7, 23, 39, ...).

So it's not suitable as a secure hash function.

Question 10

We briefly talked about Proof-of-Stake (PoS) as an alternative to Proof-of-Work (PoW).

Part A

a) In PoW, miners expend energy. What do validators in a PoS system “stake” or put at risk to participate in block creation?

In Proof-of-Stake, validators do not burn electricity like miners do in PoW. Instead, they lock up (“stake”) some of the blockchain’s native coins/tokens as collateral in order to participate in proposing/validating blocks. That staked amount is at risk: if a validator misbehaves (for example tries to validate conflicting blocks) or breaks the rules, the protocol can penalize them by taking away part of their stake (slashing).

Part B

b) Read up on the advantages and criticisms of PoS systems, and mention one for each here.

One advantage of PoS:

It is much more energy-efficient than PoW because it does not require constant brute-force hashing and massive power usage.

One criticism of PoS:

It can lead to centralization / “rich get richer”, because validators with more coins can stake more, get chosen more often, and earn more rewards, which may concentrate control in large holders or big staking pools over time.